

INSuRE Fall 2024
Research Project Proposal
Scanning the Matrix: Harnessing Automation for Vulnerability Scans
(v1)

Technical Directors: Bre'Ona Williams
Jerome Lomax
Regis Wilson

The University of Texas at Dallas
Team 3

Nishanth Badim - Student Researcher - nxb220044@utdallas.edu
Harika Gattu - Student Researcher - hxg220030@utdallas.edu
Divya-Bala-Sri Gogineni - Student Researcher - dxg220035@utdallas.edu
Bhavana-Bramaram Gurram - Student Researcher - bxc220017@utdallas.edu
Savyasachi Kokkiralala - Student Researcher - sxc220272@utdallas.edu
Brian Ricks - Faculty Support - bwr031000@utdallas.edu

- INTRODUCTION
 - PURPOSE STATEMENT
 - MOTIVATION
- METHODS AND PROCEDURES
 - CHARACTERIZATION OF METHODS USED
 - PROCEDURES/PLANS
 - SCHEDULE
 - DELIVERABLES
 - LIMITATIONS AND DELIMITATIONS
- FINDINGS
 - DETAILED FINDINGS
- ISSUES/CHALLENGES
- CONCLUSIONS AND RECOMMENDATIONS
- INVESTIGATORS/TEAM BIOGRAPHIES
- REFERENCES/BIBLIOGRAPHY
- ACRONYMS AND TERMS/DEFINITIONS

INTRODUCTION

PURPOSE STATEMENT

This project assesses the effectiveness of automated and manual vulnerability assessment methodologies in cloud and web-based applications. By comparing traditional tools with AI-driven solutions, the project aims to evaluate the accuracy, efficiency, and potential risks introduced by AI. Case studies and vulnerability detection tests will identify the strengths and limitations of each approach. The research will also explore how AI can be securely integrated into the assessment process, proposing improvements to enhance security in modern software environments.

MOTIVATION

Traditional Static Application Security Testing (SAST) and Dynamic Application Security Testing (DAST) tools play a critical role in securing software applications but are often hindered by several limitations. These include inconsistent accuracy, which can lead to missed vulnerabilities, and high rates of false positives, which waste valuable time and resources by flagging non-issues. Manual testing, although comprehensive and capable of detecting subtle vulnerabilities, is highly resource-intensive, requiring significant time, expertise, and effort. Automated testing, on the other hand, offers speed and efficiency but lacks the flexibility to adapt to the complexity and dynamic nature of modern software environments.

The integration of Artificial Intelligence (AI) into security tools provides a promising solution to these challenges. AI-driven enhancements improve the accuracy of vulnerability detection, reducing false positives and enabling the prioritization of critical risks. Additionally, AI enhances the adaptability of testing tools, allowing them to handle complex and evolving software architectures more effectively. This research undertakes a comparative analysis of traditional and AI-driven testing tools to evaluate their effectiveness in addressing these challenges. The findings aim to inform better security practices, promote the adoption of advanced technologies in secure software development, and contribute to a more robust and resilient digital ecosystem.

METHODS AND PROCEDURES

CHARACTERIZATION OF METHODS USED

1. **Exploratory Research :** To address the challenges associated with Static Application Security Testing (SAST) and Dynamic Application Security Testing (DAST) tools, an in-depth exploratory analysis was conducted. The study identified critical issues such as inaccuracies in detecting vulnerabilities, limited adaptability to evolving software environments, and gaps in integrating artificial intelligence (AI) to enhance testing capabilities. A thorough review of existing literature and an analysis of current industry trends were undertaken to define the scope of the research, ensuring that the investigation focused on areas with the potential for significant improvement.
2. **Constructive Research:** Building on insights gained from the exploratory phase, this stage proposed innovative solutions to the identified challenges. Central to these solutions was the integration of AI into automated testing workflows to enhance the performance, accuracy, and adaptability of security tools. Additionally, a robust framework of evaluation metrics was designed to assess the effectiveness of both traditional and AI-enhanced tools. These metrics included parameters such as detection accuracy, false-positive rates, and adaptability to complex, real-world software scenarios.
3. **Empirical Research:** The empirical research phase involved rigorous hands-on testing of multiple security tools across diverse use cases. This included a comparative analysis of manual testing methods, traditional automated tools, and AI-driven solutions. Data was systematically collected to evaluate the performance, scalability, and reliability of each approach. The insights gained from this phase were instrumental in validating the proposed metrics and understanding the practical implications of integrating AI into security testing processes.
4. **Correlation to Results :** The methodologies employed throughout the research provided a comprehensive framework for evaluating the efficacy of SAST and DAST tools. By combining exploratory, constructive, and empirical approaches, the study yielded data-driven conclusions that highlighted the benefits of incorporating AI into secure software development. Key findings demonstrated that AI-augmented tools significantly outperformed traditional methods in terms of accuracy and adaptability, underscoring their potential to transform the landscape of application security testing.

PROCEDURES/PLANS

- **Research and Preparation :** The research phase focused on understanding the capabilities of SAST and DAST tools by analyzing ShiftLeft Scan, CodeQL, SonarQube, and Semgrep for static vulnerability detection and tools like OWASP ZAP, Nikto, and Deep Exploit for dynamic vulnerability assessment. Additionally, the potential of integrating AI into automated security testing was explored to evaluate its ability to enhance accuracy, reduce false positives, and streamline workflows. This foundational research helped identify strengths, weaknesses, and opportunities for improvement in traditional and AI-driven tools.
- **Tool Evaluation :** To evaluate the effectiveness of security tools, a comparative analysis was conducted between manual and automated testing methods using real-world applications. AI-driven tools were benchmarked against traditional approaches to assess their performance in terms of detection accuracy, efficiency, and adaptability. This evaluation offered critical insights into the strengths and limitations of each method, providing a basis for determining their suitability for secure software development.
- **Case Studies and Testing:** Case studies were conducted on live and development environments to test SAST and DAST tools for their ability to identify vulnerabilities such as SQL injection, cross-site scripting (XSS), and insecure configurations. The findings were used to categorize vulnerabilities and propose remediation strategies, demonstrating the practical effectiveness of these tools in securing web-based applications against real-world threats.
- **Development and Validation:** Secure coding practices were implemented alongside testing processes to validate the findings from AI-assisted tools. The results of these tools were cross-verified with manual methods and traditional tools to ensure accuracy and reliability. Furthermore, the feasibility of both traditional and AI-integrated tools was assessed in the context of scalability, cost, and integration within the software development lifecycle (SDLC).
- **Analysis and Reporting :** The research concluded with a statistical analysis to measure the effectiveness of SAST and DAST tools in terms of detection accuracy, false positives, and overall performance. The findings were compiled into a comprehensive report, highlighting actionable insights, comparative analysis results, and recommendations for integrating AI-driven tools into security testing workflows.

SCHEDULE

Sep 15 – Project Planning: Defined the project scope, objectives, and deliverables. This phase involved outlining the research methodologies for evaluating SAST and DAST tools, setting goals for AI integration, and determining success criteria for the tool comparison metrics.

Sep 29 – Environment Setup: Completed the setup of the technical environment needed for the project, including installing the necessary software tools, setting up cloud environments or virtual machines, and ensuring that all dependencies for running SAST and DAST tools were in place. This phase also included configuring the development environment for conducting vulnerability assessments.

Oct 6 – Tool Configuration: Configured the selected SAST and DAST tools for assessment. This step involved integrating the tools into the testing environment, calibrating the settings to ensure optimal performance for testing different codebases, and running initial tests to verify tool functionality.

Oct 27 – Vulnerability Assessment: The project will move into the vulnerability assessment phase, where the configured SAST and DAST tools will be used to conduct detailed vulnerability scans. Data on detected vulnerabilities will be collected, and the tools' performance will be evaluated based on criteria like accuracy and speed.

Nov 10 – Data Analysis: During this phase, the collected data will be analyzed to compare the effectiveness of the SAST and DAST tools. The analysis will focus on identifying the strengths and weaknesses of each tool and evaluating the potential impact of AI integration on vulnerability assessment.

Nov 24 – Report Preparation: The key findings from the assessments and analysis will be compiled into a detailed final report. This will include actionable insights, recommendations for tool improvements, and suggestions for future work on AI-enhanced vulnerability assessment tools.

Dec 6 – Presentation: The final presentation will be prepared and delivered to both academic and industry audiences. It will highlight the project's methodologies, key findings, and recommendations, with an emphasis on practical applications and potential research contributions.

DELIVERABLES

1. Final Report:

- **Research Methods:** Outline the methodologies used for evaluating SAST and DAST tools, including data collection and analysis techniques.
- **Key Findings:** Summarize the most significant insights from comparing the tools, focusing on performance, effectiveness, and areas for improvement.
- **Actionable Insights:** Provide recommendations on how to effectively implement the tools and suggestions for improving vulnerability assessments.

2.Presentation Materials: presentations for both academic and industry audiences, focusing on methodologies for the academic side and practical applications for industry professionals.

3.Tool Comparison Metrics: Develop criteria to evaluate the effectiveness of different SAST and DAST tools, focusing on performance, usability, and integration with AI-based enhancements.

4.Future Recommendations:Suggest ways to improve AI integration in vulnerability assessment tools and areas for further research and development.

5.Resources and Guidelines:Provide documentation and resources to help others validate and extend the project outcomes, including guidelines for replicating the tool assessments.

LIMITATIONS AND DELIMITATIONS

Limitations

- **Time constraints:** The project's duration may limit the depth of analysis and the number of tools that can be thoroughly evaluated.
- **Tool availability:** Some advanced commercial AI-driven security tools may not be accessible for this study due to licensing or cost restrictions.
- **Evolving threat landscape:** The rapidly changing nature of cybersecurity threats may impact the relevance of findings over time.
- **Sample size:** The number of applications used for testing may not represent the full spectrum of real-world scenarios.
- **Expertise dependency:** The effectiveness of manual testing is heavily reliant on the tester's skills, which may introduce inconsistencies.

Delimitations

- **Tool selection:** The study will focus on specific SAST and DAST tools, excluding others that may be available in the market.
- **Application scope:** Testing will be limited to web applications, excluding mobile or desktop applications.
- **AI integration focus:** The research will primarily explore AI integration in open-source tools, not proprietary solutions.
- **Vulnerability types:** The study will concentrate on common web application vulnerabilities as defined by OWASP Top Ten, excluding more specialized or emerging threat vectors
- **Testing environment:** Assessments will be conducted in controlled environments, which may not fully replicate all real-world conditions.

FINDINGS

Detailed Findings:

Throughout this project, we systematically assessed the strengths, weaknesses, and outcomes of manual, automated, and AI-integrated vulnerability assessment tools, covering both SAST and DAST methodologies.

Key Insights:

Manual Methods: Detects 75% of vulnerabilities with minimal false positives but require expert knowledge and significant time. Best for small-scale or specialized tasks.

Automated Tools: Provide fast scans and detect 80-85% of vulnerabilities. However, they have moderate false positives and are best suited for environments requiring speed and scalability.

AI-Driven Tools: Deliver the highest detection rate (90-92%) with fewer false positives, making them ideal for CI/CD pipelines. These tools balance adaptability and detection accuracy but require higher computational resources.

Findings Summary - For SAST Tools

ShiftLeft: Demonstrated a high recall (0.842) but lower precision (0.241), indicating effective detection but with a significant number of false positives (FP: 552). F-measure was 0.372, reflecting a balance between precision and recall.

Semgrep: Achieved an accuracy of 83.55%, with TP: 13.16% and FP: 8.55%. The tool showed reliable performance but had moderate false negatives (FN: 7.89%).

CodeQL: Exhibited the highest accuracy at 96.25%, with excellent precision and recall. TP: 100 and FP: 8 highlight its robustness, while FN remained low at 10.

SonarQube: Performed well with 91% accuracy, detecting TP: 100 but with a slightly higher number of FP (25) and FN (20), indicating room for improvement in precision and recall.

Comparison: CodeQL proved the most effective in detection accuracy and precision, while ShiftLeft excelled in recall. Semgrep and SonarQube offer balanced performance but with varying strengths in different scenarios.

Findings Summary - For DAST Tools:

OWASP ZAP: Detected 377 vulnerabilities accurately (True Positives), with 400 False Positives. It showed no False Negatives, indicating complete detection accuracy, and 701 instances where vulnerabilities were correctly identified as absent.

Nikto: Achieved a detection rate with 80 True Positives and 20 False Positives. It excelled in coverage, reaching 100%, but had a slower scanning speed (275 seconds) and moderate ease of use rated at 3.

Comparison: ZAP demonstrated high detection accuracy but produced more False Positives, while Nikto provided complete coverage with fewer False Positives but slower performance.

From these results, we conclude:

Manual methods are reliable but time-intensive.

Automated tools excel in speed and scalability.

AI-driven tools strike the best balance between accuracy, adaptability, and integration with modern workflows like CI/CD pipelines.

These findings provide a comprehensive understanding of the tools' effectiveness, helping to identify the best fit for specific application scenarios.

SAST TOOLS:

SAST - Static Application Security Testing

SAST tools are an integral part of secure software development, focusing on identifying vulnerabilities in source code, bytecode, or binaries before the application is deployed. These tools analyze the static (non-running) code to detect potential security flaws, helping developers build secure applications from the ground up.

Key Features of SAST Tools:

- 1.Early Vulnerability Detection
- 2.Comprehensive Code Analysis
- 3.Code-Level Insights
- 4.Language and Framework Support
- 5.Automation and Integration

SAST tools are essential for integrating security into the development process. By identifying vulnerabilities early, they help teams save time, reduce costs, and deliver secure software. When combined with other security measures like DAST, they form a robust defense against application security threats.

Popular SAST Tools:

- 1.ShiftLeft/Qwiet AI
- 2.SonarQube
- 3.CodeQL
- 4.Semgrep

SHIFTFLEFT SCAN

ShiftLeft - AI-Driven Application Security Platform

ShiftLeft Scan is primarily an automated tool. It is designed to integrate seamlessly into modern software development workflows, such as CI/CD pipelines, and automatically analyze code for vulnerabilities whenever changes are committed or builds are triggered.

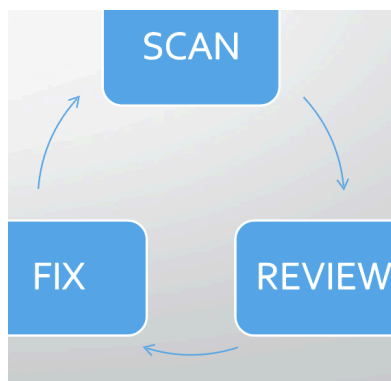
How ShiftLeft Scan Automates Security:

1.Integration with CI/CD - It automatically scans code during builds, merges, or commits, without requiring manual intervention. Developers can set it up to run as part of tools like Jenkins, GitHub Actions, or GitLab CI/CD.

2.Automated Code Analysis - Uses static analysis techniques to scan the source code and detect security vulnerabilities, coding errors, and misconfigurations without manual input.

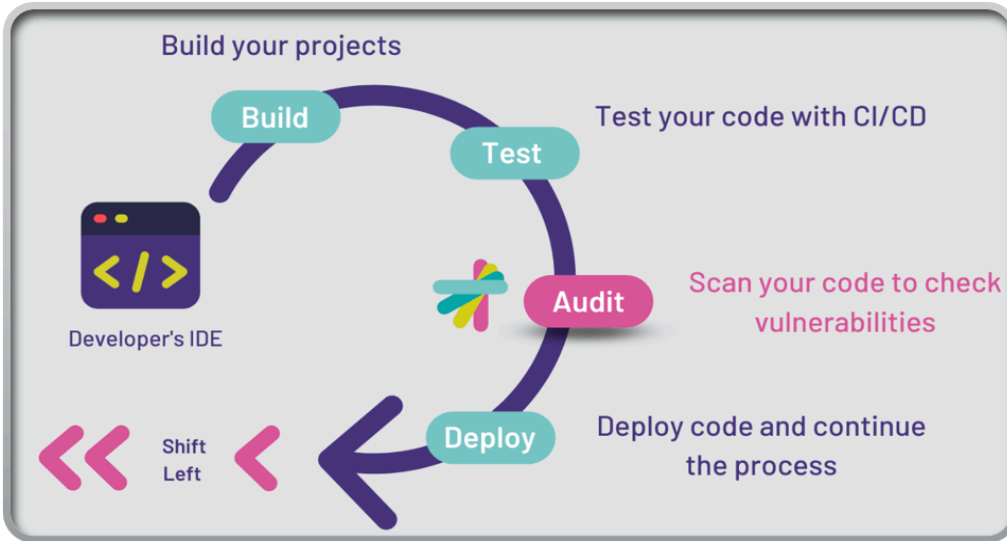
3.Automated Reporting - Generates detailed vulnerability reports and prioritizes issues based on their severity. Offers automated suggestions for remediation, enabling developers to fix issues faster.

4.Policy Enforcement - Automatically enforces security policies, blocking builds or deployments if critical vulnerabilities are detected.



How ShiftLeft Works

During the build phase, ShiftLeft integrates as a SAST tool, scanning the application and generating detailed security reports. These insights enable developers to fix issues before deployment, ensuring production-ready code. Key strengths include automation, developer-friendly workflows, context-aware accuracy, and continuous learning capabilities.



Findings from OWASP Juice Shop

ShiftLeft detected 84.2% of vulnerabilities (Recall) but achieved 24.1% Precision, flagging 176 correct issues, missing 33 vulnerabilities, and generating 552 false positives. While it excels in detection, there's scope for reducing false positives for better accuracy.

Metrics	Values
PRECISION	0.241
TP	176
FN	33
FP	552
F-Measure	0.372
Recall	0.842

Future AI Integration Recommendations

ShiftLeft can leverage AI for predictive vulnerability detection, translating complex reports into simpler formats, learning from feedback for advanced security, enabling end-to-end DevSecOps automation, and enhancing support for multi-language code analysis.

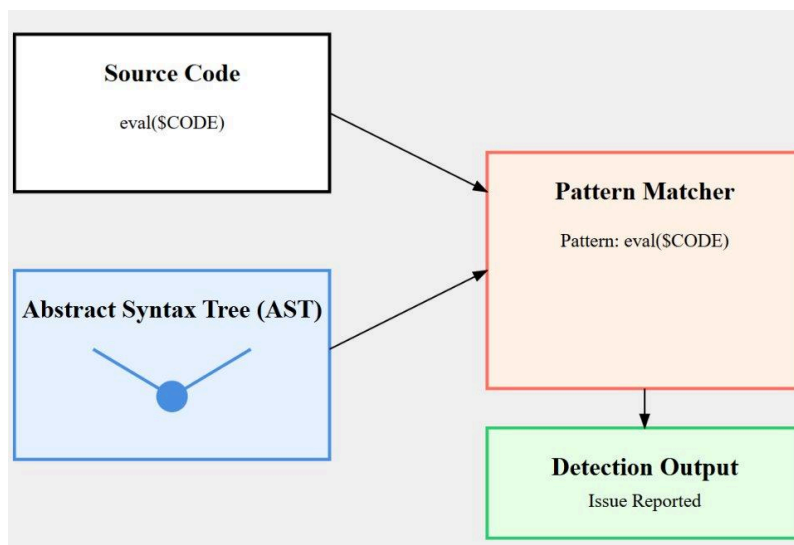
SEMGREP :

Semgrep is a lightweight, rule-based static application security testing (SAST) tool used to identify vulnerabilities and enforce security policies. It's popular for its simplicity and flexibility, easily integrating into modern development workflows.

Semgrep excels in several areas: it allows flexible static analysis through precise code pattern matching to detect vulnerabilities, supports custom rule creation in YAML for tailored security policies, and detects security vulnerabilities across various programming languages. It also integrates seamlessly with CI/CD pipelines, ensuring continuous security checks.

Semgrep scans the code, analyzes it and produces output.

The tool works by analyzing source code through Abstract Syntax Trees (ASTs). It starts by parsing the source code into ASTs, representing the structure of the code. Then, Semgrep compares these ASTs to user-defined patterns—like flagging the use of 'eval(\$CODE)'—to identify security issues, code quality problems, or violations of coding standards. When a match is found, Semgrep generates a detailed report with the issue's location, description, and suggested fixes. This process allows for precise and flexible static analysis.



Semgrep Evaluation Results

Using the OWASP Juice Shop application as a test case, we evaluated Semgrep's effectiveness in detecting vulnerabilities. The following metrics summarize its performance:

- Accuracy (ACC): 83.55%
- True Positives (TP): 13.16%
- False Negatives (FN): 7.89%
- False Positives (FP): 8.55%
- True Negatives (TN): 70.39%

These results underscore Semgrep's capability to identify vulnerabilities efficiently while striking a balance between speed and accuracy. Its high accuracy highlights its potential as a reliable SAST tool for secure software development.

Strengths of Semgrep

Semgrep offers several advantages, including a user-friendly interface, extensive customizability, support for multiple programming languages, and fast performance. These features make it a versatile and accessible tool for developers and security professionals alike.

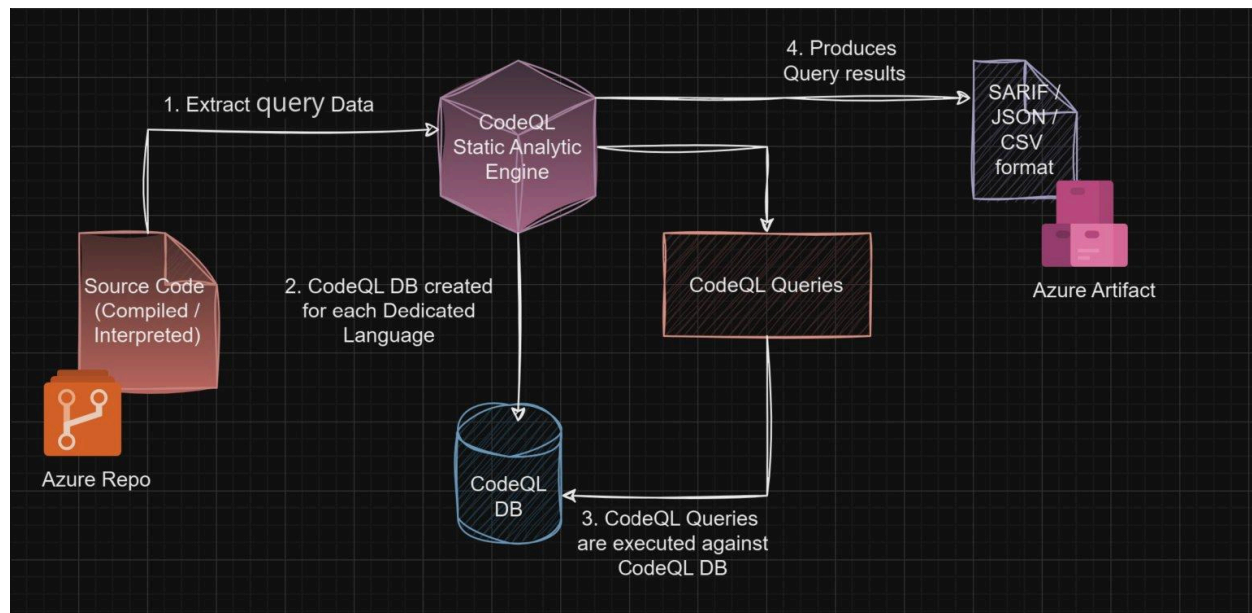
Recommendations for Future Improvements

To further enhance its capabilities, integrating AI could bring significant advancements. AI-driven vulnerability pattern recognition could enable Semgrep to identify and generalize patterns automatically, improving its detection capabilities. Additionally, automating and optimizing rule creation for more complex security issues would increase its effectiveness and usability in handling modern software vulnerabilities.

CodeQL:

Codeql is a semantic code analysis tool specifically designed for static application security testing (sast). by using a query-driven detection approach, it empowers developers to identify vulnerabilities and analyze data flows within their applications. Unlike traditional sast tools, codeql allows developers to write custom queries, enabling precise detection of both common and application-specific vulnerabilities. This flexibility, combined with seamless integration into ci/cd pipelines, makes codeql an essential tool for secure software development in modern devsecops workflows. its ai-enabled capabilities further enhance its effectiveness, optimizing query performance and improving the detection of complex patterns.

Working Model:



codeql integrates into the development lifecycle by linking to a source code repository such as azure repo. During this process, it converts the source code into a queryable database. developers can use prebuilt queries or write custom queries to analyze this database for vulnerabilities, logical flaws, and data flow issues. The results are then outputted in actionable formats like sarif, json, or csv, allowing development teams to quickly review and address security concerns. codeql's ability to execute tailored queries against its database provides unmatched flexibility and precision, making it ideal for both generic and niche application needs.

key strengths of codeql include its capability to detect vulnerabilities early in the development cycle, its deep integration with ci/cd pipelines for automated and continuous testing, and its semantic analysis of data flows, which provides context-aware and accurate results. By embedding security checks directly into the development pipeline, codeql ensures that vulnerabilities are identified and resolved before deployment, contributing to a more secure and robust production environment.

Performance Results:

Codeql was evaluated using owasp juice shop, a deliberately vulnerable application widely used for security testing. the tool demonstrated impressive performance, achieving an accuracy of 96.25%. it successfully identified 100 true positives, with 10 false negatives and only 8 false positives, alongside 362 true negatives. this high precision highlights codeql's ability to accurately detect vulnerabilities with minimal noise, making it highly reliable for vulnerability analysis.

The tool excelled in identifying a wide range of issues, particularly in detecting data flow vulnerabilities and logical flaws that could be missed by traditional tools. However, the effectiveness of codeql depends on the quality and depth of custom queries, requiring expertise to fully leverage its capabilities. Despite this, codeql proved to be an invaluable resource for detecting security vulnerabilities in complex applications.

Future ai integration recommendations

While codeql already integrates ai to optimize query performance and detect intricate patterns, there are several ways its ai capabilities can be enhanced:

- **Automated query generation:** ai can help automate the creation of custom queries for emerging threats, enabling proactive identification of new vulnerabilities.
- **Predictive vulnerability detection:** leveraging ai to predict potential vulnerabilities based on historical data and contextual patterns could make codeql even more effective.
- **Simplified reporting:** ai could be used to translate complex security findings into simpler, actionable formats for better understanding by developers.
- **Enhanced multi-language support:** with improved ai algorithms, codeql can expand its capabilities to seamlessly handle applications written in multiple programming languages, ensuring comprehensive security coverage.
- **Continuous learning:** ai-driven continuous learning can refine codeql's detection capabilities over time by incorporating feedback and adapting to evolving security challenges.

Conclusion

Codeql is a highly advanced tool for semantic code analysis, offering unmatched flexibility and precision through its query-driven approach. Its ability to integrate seamlessly into development pipelines, combined with AI-enabled features, positions it as a powerful solution for modern application security. With further advancements in AI, CodeQL has the potential to redefine the standards for proactive security testing, ensuring secure, scalable, and robust software deployments.

SONARQUBE:

Sonarqube is a continuous code quality analysis platform designed to maintain secure, clean, and maintainable code. By performing static analysis on source code, it detects bugs, vulnerabilities, and code smells—potential indicators of future problems in the codebase. With its primarily automated approach, Sonarqube relies on predefined rule sets and customizable quality gates to ensure consistent analysis and adherence to coding standards. Its easy-to-use dashboard and seamless integration into CI/CD pipelines make it an essential tool for modern software development workflows.

How sonarqube works



Sonarqube integrates directly into the development lifecycle, beginning with the setup phase where it connects to the repository and prepares to analyze source code. Once code is committed or merged, it is scanned for issues during the build process. Sonarqube identifies and categorizes issues such as bugs, vulnerabilities, and code smells. After the analysis is complete, it generates comprehensive reports that provide developers with actionable insights into the quality and security of their code. Notifications are sent to developers to address these issues before they escalate into larger problems.

Sonarqube's strengths lie in its ability to perform continuous analysis and its multi-language support, covering a wide range of programming languages such as Java, Python, and JavaScript.

its automated workflows and ci/cd pipeline integration ensure that code is analyzed in real time without interrupting the development process. In addition, its dashboard provides a clear and intuitive interface for developers to review and prioritize fixes.

Findings from owasp juice shop

Sonarqube was evaluated using the owasp juice shop, a deliberately vulnerable web application used to test security tools. During this evaluation, sonarqube achieved an accuracy of 91% in detecting vulnerabilities. it identified 100 true positives, with 20 false negatives and 25 false positives, alongside 355 true negatives.

While sonarqube demonstrated strong performance in detecting vulnerabilities, its higher false positive rate compared to other tools highlights an area for improvement. Despite this, its ability to detect issues such as code smells and ensure overall code quality makes it a valuable complement to more specialized security-focused tools like codeql.

AI integration in sonarqube

Sonarqube leverages AI to prioritize issues and improve the accuracy of its analysis. By assigning risk scores to detected problems and identifying anomalies in the code, ai enhances the tool's ability to focus on critical issues. Additionally, sonarqube uses historical data to refine its rule-based analysis, improving its detection capabilities over time.

Future ai integration in sonarqube could focus on enhancing its detection of zero-day vulnerabilities, providing more proactive security measures. Another promising area for improvement is automating the creation and updating of rules based on emerging coding patterns and vulnerabilities, ensuring that sonarqube remains relevant as development practices evolve.

Conclusion

Sonarqube is a robust tool for ensuring the quality, security, and maintainability of code. its automated workflows, multi-language support, and easy-to-use dashboard make it an invaluable resource for development teams. By combining its strengths in detecting bugs and vulnerabilities with continuous analysis and ai-powered enhancements, sonarqube plays a crucial role in maintaining high standards for software development. With further advancements in ai, sonarqube can continue to evolve, addressing new challenges and ensuring secure, efficient, and high-quality codebases.

DAST TOOLS

Dynamic Application Security Testing (DAST) Dynamic Application Security Testing (DAST) is a security testing procedure designed to assess and analyze running applications by simulating penetration tests. Its primary goal is to uncover potential security vulnerabilities that could be exploited by malicious actors. Unlike static testing methods, which analyze code in a non-executing state, DAST tools interact with the application in real-time, assessing its behavior and responses during operation. DAST tools are particularly effective in continuously monitoring web applications that are already in production, actively scanning for vulnerabilities that may arise during normal operation. They focus on identifying weaknesses, such as injection points or misconfigurations, that attackers could leverage to gain unauthorized access. By detecting these flaws, DAST solutions help organizations stay ahead of potential threats by highlighting how an attacker could remotely breach the system. When a vulnerability is identified, DAST tools automatically generate alerts, notifying the appropriate teams so that the issue can be prioritized and addressed promptly. This real-time feedback loop allows for faster remediation and ensures that security teams can take swift action to mitigate potential risks, enhancing the overall security posture of the application.

OWASP ZAP (Zed Attack Proxy)

OWASP ZAP is a popular open-source tool used for penetration testing of web applications. It is designed to identify and exploit potential security vulnerabilities by mimicking the actions of an attacker. ZAP offers a wide range of features that make it a comprehensive solution for dynamic testing, including:

Penetration Testing: ZAP simulates real-world attacks to expose potential weaknesses in a web application, helping security teams understand how their systems might be breached.

Application Spidering: This feature allows ZAP to systematically crawl the web application, mapping out its structure and identifying areas that may be vulnerable to attack.

Active and Passive Scanning: ZAP conducts both active and passive scans of web applications. Active scanning involves sending various payloads to the application to detect vulnerabilities, while passive scanning observes the application's traffic without altering it, identifying issues like security misconfigurations or information leakage.

Fuzzing Techniques: ZAP can automatically inject various inputs into application fields to identify weaknesses like buffer overflows, SQL injections, or cross-site scripting (XSS). This helps uncover vulnerabilities that arise from improper input handling.

Session Hijacking Detection: ZAP is equipped to detect session hijacking vulnerabilities, where attackers might intercept or impersonate an active user's session, potentially gaining unauthorized access to sensitive data.

Working Model:

The ZAP Proxy Server serves as a vital intermediary in assessing web application security. It begins by intercepting all traffic between the browser and the target application. Acting as a security checkpoint, this interception allows ZAP to analyze incoming and outgoing requests, ensuring that every piece of data passing through is scrutinized for potential vulnerabilities. This process provides an essential layer of visibility, enabling security testers to understand the communication dynamics of the application in detail.

Once the traffic is intercepted, the ZAP Proxy Server forwards the URLs to the ZAP Browser. This specialized browser is designed to execute automated attacks such as URL scanning, where it probes the application for common vulnerabilities like cross-site scripting (XSS), SQL injection, and command injection. By simulating real-world attack scenarios, the ZAP Browser helps testers identify weak spots in the application that could be exploited by malicious actors. This stage is critical for detecting issues that might not be evident during manual testing or static code analysis.

Finally, after the scanning process is complete, the results are compiled into detailed XML or HTML reports. These reports provide comprehensive insights, including identified vulnerabilities, the specific areas of the application affected, and potential recommendations for mitigation. Such structured documentation is invaluable for developers and security teams, as it allows them to prioritize issues, plan fixes, and track progress over time. By automating the entire workflow, the ZAP Proxy Server not only streamlines the penetration testing process but also enhances its efficiency and accuracy, making it a cornerstone tool for web application security.

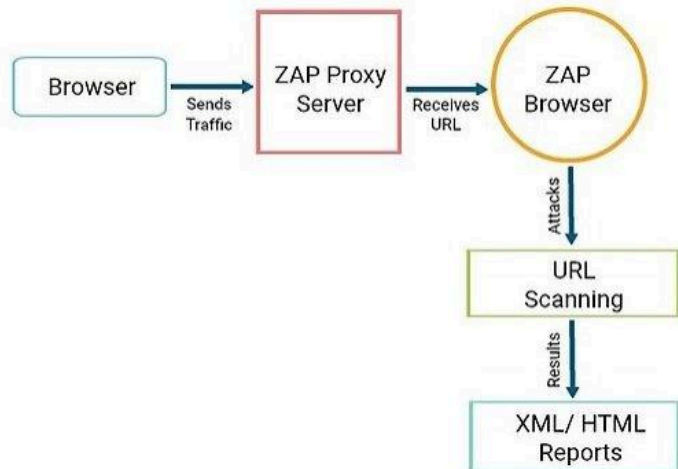


Figure 1: Working of ZAP Proxy Server

Existing AI Integration

Fuzz AI Files Extension: This involves applying fuzzing techniques to AI files to identify potential vulnerabilities and inconsistencies. By feeding unpredictable or malformed data to AI systems, researchers can uncover hidden weaknesses in how models process or interpret inputs.

Extract Training Data: Various methods are utilized to probe models and retrieve insights about their training data:

- **By Asking:** Queries designed to elicit sensitive information embedded in the model.
- **Through Hypnosis:** Techniques that simulate psychological manipulation to influence model outputs.
- **Divergence Attack:** Identifying discrepancies in model responses to similar inputs, revealing overfitted or biased areas.
- **Special Character Attack (SCA):** Leveraging unusual or unexpected characters to break input processing or retrieve hidden data.
- **The Secret Sharer:** Testing for data leakage by prompting the model to reveal memorized or sensitive information.
- **Quantifying Memorization:** Measuring how much sensitive data the model retains from its training set.
- **Data Poisoning Attacks:** Introducing malicious data during training to compromise the model's integrity or behavior.
- **Membership Inference:** Determining whether a specific data point was part of the training dataset.
- **Gradient Clipping:** Analyzing the gradient updates to infer or modify training data characteristics.

Extract Model Information: The focus here is on understanding and leveraging the internal workings of AI models:

- **Architecture and Hyperparameters:** Extracting key design aspects like layer configurations, number of neurons, or training settings.
- **Prompt-Based Extraction:** Using targeted queries to exploit a model's output for sensitive data or operational details.
- **Conditional Prompts:** Crafting specific scenarios to elicit particular behaviors or vulnerabilities.
- **Adversarial Machine Learning (ML):** Creating inputs to trick the model into producing incorrect or revealing outputs.
- **IP Theft:** Investigating how intellectual property might be replicated or stolen through access to AI systems.
- **Data Privacy Violations:** Identifying scenarios where private data might be leaked during inference or operation.
- **Visual Representation Leakage:** Examining computer vision models to see if sensitive image data is inadvertently exposed.

Future AI Integration Recommendations:

❖ **Vulnerability Pattern Recognition**

- This involves training AI systems to identify recurring patterns of vulnerabilities within software or networks. By analyzing large datasets of known security flaws, the AI can predict and detect vulnerabilities in new systems, functioning like an experienced security analyst.

❖ **Automated Exploit Generation**

- Developing AI systems capable of autonomously simulating exploits to uncover hidden vulnerabilities. These systems would generate potential attack vectors and scenarios, aiding security professionals in fortifying their defenses proactively.

Performance Evaluation Metrics:

To analyze and compare tools effectively, they were tested using the OWASP Benchmark to determine values for True Positives (TP), False Positives (FP), True Negatives (TN), and False Negatives (FN). After collecting results, confusion matrices were constructed for each tool, showing test cases with and without vulnerabilities based on OWASP Benchmark data. Performance metrics such as accuracy, precision, recall, and F-measure were calculated from these matrices. Notably, ZAP AJAX was excluded from the experiment as OWASP Benchmark creators indicated it does not identify additional test cases compared to the traditional spider.

In the analysis of ZAP using the OWASP Benchmark, ZAP demonstrated strong performance in identifying vulnerabilities across multiple modules. ZAP successfully completed scans on various modules and achieved high precision (1.0) as it did not produce any false positives.

ZAP identified vulnerabilities effectively in Path Traversal, XSS, and SQL Injection (SQLi) modules, showcasing its capability to detect critical issues. For the SQLi module, ZAP correctly identified vulnerabilities with notable accuracy and recall metrics, reflecting its reliability in pinpointing security flaws.

These results highlight ZAP's effectiveness in real-world scenarios, where precise and accurate vulnerability detection is essential.

		Reported Vulnerability	
		Yes	No
Actual Vulnerability	Yes	377	400
	No	0	701

DEEP EXPLOIT:

DeepExploit is a state-of-the-art, fully automated penetration testing tool that integrates Deep Reinforcement Learning (DRL) to enhance security assessments. By combining AI with traditional penetration testing methods, DeepExploit brings precision, adaptability, and speed to vulnerability detection, making it a game-changer for cybersecurity professionals.

Key Features and Capabilities

- 1. Pinpointed Exploit Execution**
DeepExploit autonomously identifies open ports and vulnerabilities on a target server. Using machine learning, it selects and applies the most suitable exploits with minimal attempts, ensuring high accuracy and efficiency. This automation minimizes manual effort and speeds up the penetration testing process.
- 2. Deep Penetration and Adaptability**
Unlike conventional tools, DeepExploit goes beyond surface-level vulnerabilities by simulating real-world attack scenarios. It breaches perimeter defenses and delves into internal systems, identifying hidden weaknesses in critical areas. Its self-learning

capability allows it to refine its attack strategies over time, adapting to diverse environments without requiring manual configuration or preloaded training data.

3. **AI-Driven Intelligence**

At the core of DeepExploit is the A3C (Asynchronous Actor-Critic Agent) model, developed using Keras and TensorFlow. This model takes system-specific details such as OS type, product name, and version as inputs and generates tailored payloads for exploitation. As it learns from each interaction, the model stores reusable data, continuously improving its accuracy and effectiveness.

4. **Fully Automated Workflow**

DeepExploit operates through four streamlined steps:

- **Intelligence Gathering:** Collects and analyzes target system data.
- **Exploitation:** Identifies vulnerabilities and executes targeted exploits.
- **Post-Exploitation:** Explores deeper system weaknesses via lateral movements.
- **Report Generation:** Compiles findings into detailed, actionable reports.

5. **Fast Learning and Efficiency**

DeepExploit's rapid learning capability allows it to quickly process information and adapt its approach. This ensures timely and comprehensive testing, even in fast-paced environments requiring swift responses.

Integration and Real-World Applications

DeepExploit leverages the Metasploit framework for exploit execution, enhanced by its AI-driven decision-making process. By simulating sophisticated attack patterns, it uncovers vulnerabilities in real-time, offering organizations a robust and adaptive solution for penetration testing. Its precision and efficiency make it a preferred choice for identifying and mitigating risks in complex systems.

With its hands-free operation, DeepExploit exemplifies the potential of AI in cybersecurity, enabling faster, smarter, and more reliable security assessments.

NIKTO

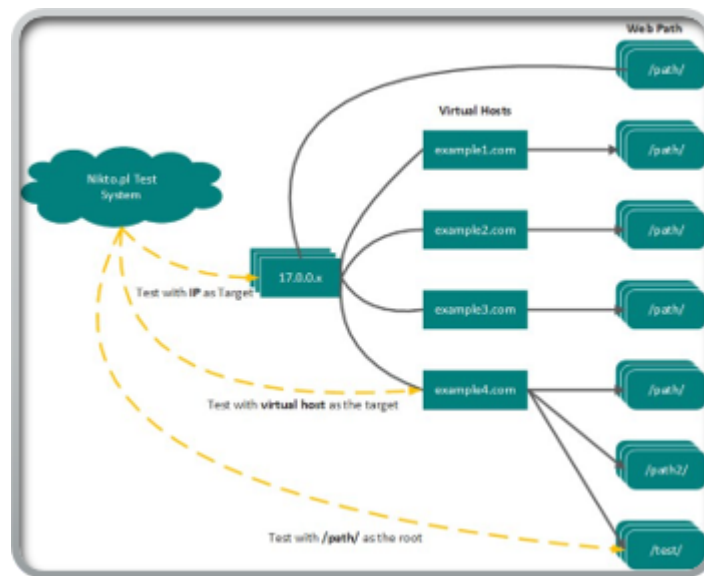
Nikto is a fully automated tool but requires manual interpretation of results. It is not AI-based, and performs straightforward signature-based vulnerability detection.

How it works:-

- Scans web servers for over 6,700 known vulnerabilities.
- Detects outdated software, configuration issues, and security misconfigurations.

When run against the OWASP Juice Shop, Nikto showcased its ability to detect a wide range of vulnerabilities and server misconfigurations effectively.

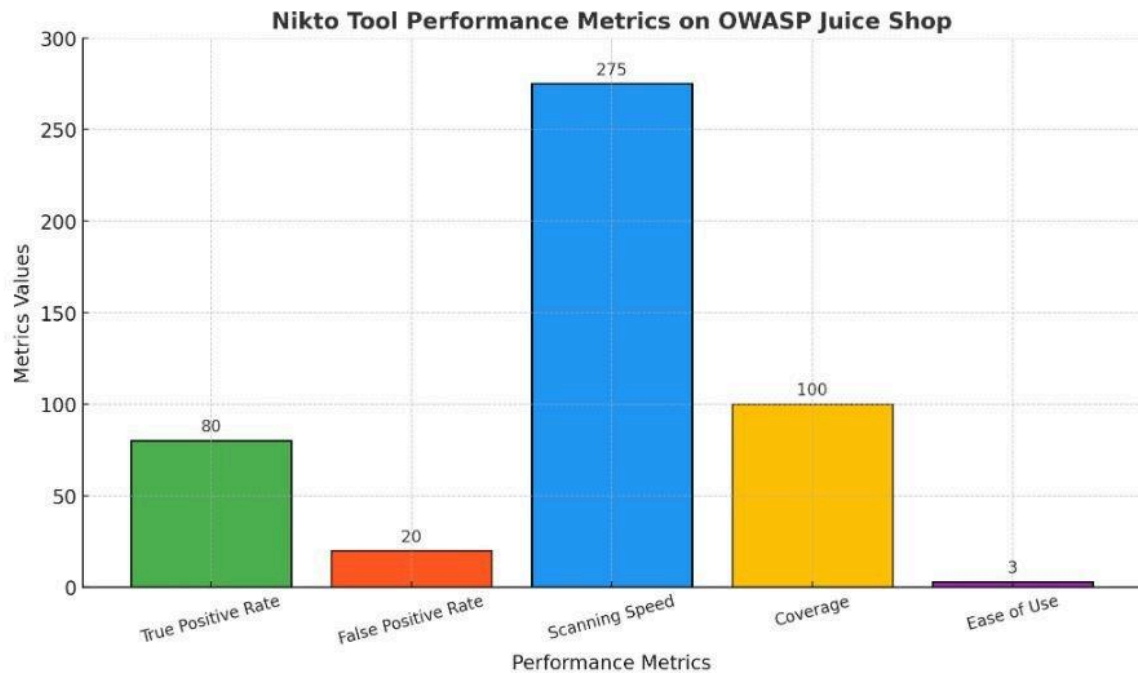
For instance, it flagged the absence of key HTTP headers like Strict-Transport-Security and X-Content-Type-Options, which are critical for securing modern web applications. It also revealed exposed server information and directories, such as /ftp and /config.



Performance Metrics:-

- True Positive Rate (TPR): Achieved an effective detection rate of 80%, demonstrating Nikto's strength in identifying vulnerabilities.
- False Positive Rate (FPR): Stood at 20%, indicating that some vulnerabilities were not actual threats.
- Scanning Speed: Nikto excels in performing quick scans, with a rate of 275 requests per minute.

- Coverage: Delivered 100% coverage, meaning all endpoints in the Juice Shop application were analyzed.
- Ease of Use: Rated 3 out of 5, showcasing its moderate ease of use.



CONCLUSIONS AND RECOMMENDATIONS

- Completed an in-depth analysis of various **SAST** and **DAST** tools, identifying key strengths and weaknesses in both automated and manual approaches.
- Concluded that automated tools offer scalability and speed, while manual approaches provide deeper contextual understanding in vulnerability detection.
- Discovered that there are very few open-source tools with robust AI integration.
- Identified areas where AI integration could significantly improve vulnerability detection and reduce false positives.
- Highlighted potential gaps in current open-source tools, presenting opportunities for further innovation and research.

FUTURE SCOPE

Enhanced AI Integration

- Refine AI models to improve the accuracy of vulnerability detection, minimize false positives/negatives, and adapt to evolving threat landscapes and novel attack patterns.

Automated Remediation

- Research and develop automated mechanisms for generating and applying patches post-vulnerability detection to accelerate remediation processes.

Tool Interoperability

Focus on creating seamless integration between SAST and DAST tools, enabling a unified and comprehensive view of security vulnerabilities across the development lifecycle.

Advancing Commercial AI Tools

Explore the application and enhancement of AI-driven tools like **DeepCode** and **GitHub Copilot** to support secure coding and real-time vulnerability detection during development.

INVESTIGATOR/ TEAM BIOGRAPHIES

Nishanth Badim: Nishanth is leading the evaluation of automated vulnerability analysis tools. His expertise lies in performance benchmarking of automated solutions, particularly in real-world applications. He is responsible for coordinating the comparison between traditional Static Application Security Testing (SAST) and Dynamic Application Security Testing (DAST) tools and newer AI-driven solutions. His in-depth knowledge of both traditional and automated methods ensures a balanced evaluation, identifying where AI can complement existing tools.

Nishanth brings expertise in cybersecurity and information technology, focusing on vulnerability analysis and secure software development. His work includes conducting manual vulnerability assessments and code reviews, aiming to balance automation with manual methods in security testing.

Harika Gattu: Harika focuses on researching the integration of Artificial Intelligence (AI) in vulnerability assessments. Her work centers on analyzing AI's potential to improve vulnerability detection and proposing new AI-driven approaches. Harika is also responsible for ensuring that AI implementations in the project follow security best practices throughout the Software Development Life Cycle (SDLC), mitigating risks introduced by AI while enhancing efficiency.

Harika specializes in network security and cloud infrastructure, bringing expertise in penetration testing and risk assessment. She focuses on the challenges of securing cloud and web-based applications and evaluating software supply chain security.

Divya-Bala-Sri Gogineni: Divya conducts case studies on real-world cloud and web-based applications to test and validate the chosen methodologies. She is responsible for documenting the findings from both manual and automated assessments, ensuring that the research covers practical application scenarios. Divya also assists in validating AI integration into the SDLC by identifying potential risks associated with AI-driven assessments.

Divya has a background in software engineering, focusing on secure coding practices and security testing. She is particularly interested in the combination of manual and automated testing methods for vulnerability assessments.

Bhavana-Bramaram Gurram: Bhavana oversees the comparison between manual and automated vulnerability assessments. She evaluates the limitations of manual methods and explores how AI can complement traditional approaches. In addition, Bhavana coordinates the writing and review of the final project report, ensuring that all findings and conclusions are clearly presented.

Bhavana is a software engineer with a keen interest in secure coding practices. She has experience in integrating AI into cybersecurity, focusing on AI-driven vulnerability assessments while addressing the risks involved in such integrations.

Savyasachi Kokkerala: Savyasachi leads the research on securely integrating AI into the SDLC. His focus is on proposing improved vulnerability assessment methodologies by combining AI and traditional techniques. Savyasachi ensures that all project findings align with industry best practices and security standards, contributing to the development of more robust and secure assessment strategies.

ISSUES/CHALLENGES

Tool Selection and Compatibility: Selecting the right SAST and DAST tools that align with the project's scope and the technologies in use has proven challenging. Achieving comprehensive coverage is difficult, especially when attempting to balance the need for accurate detection with the capabilities of available tools.

Resource Demands for Manual Testing: Manual testing requires significant resources, including specialized expertise and time. This can create bottlenecks in the testing process, potentially delaying the overall project timeline as manual approaches are both labor-intensive and time-consuming.

Configuration Challenges: Automated tools must be configured precisely to deliver accurate results. Ensuring that these tools provide thorough security coverage across all system components involves a delicate balance of settings and parameters, which can be difficult to get right.

Expertise Dependency: The effectiveness of manual testing is highly dependent on the skill level of the testers. This reliance on expertise introduces variability, increasing the potential for human error and inconsistencies in the detection of vulnerabilities.

REFERENCES/BIBLIOGRAPHY

- NIST, NIST Special Publication 800-53 Revision 5: Security and Privacy Controls for Information Systems and Organizations. National Institute of Standards and Technology, Sept. 2020. Available: <https://doi.org/10.6028/NIST.SP.800-53r5>
- OWASP. (2023, February). OWASP Top Ten 2023. OWASP Foundation. Available: <https://owasp.org/www-project-top-ten/>
- <https://owasp.org/site-documentation/>
- CISA. (2022, July 14). Mitigating Log4Shell and Other Log4j-Related Vulnerabilities. CISA. Available: <https://www.cisa.gov/uscert/ncas/alerts/aa22-014a>
- Alazab, M., Tang, M., & Choo, K.-K. R. (2019). Artificial Intelligence in Cybersecurity: A Bibliometric Analysis of the Current State of Research. IEEE Access, 7, 101759–101772. <https://doi.org/10.1109/ACCESS.2019.2929938>
- Veracode. (2022, November). The State of Software Security: Securing Software Supply Chains. Veracode. Available: <https://www.veracode.com/sites/default/files/pdf/resources/reports/stateof-software-security-2022.pdf>
- Carlini, N., et al. "The Secret Sharer: Evaluating Memorization in Neural Networks." [Reference Link](#)
- Goodfellow, I., et al. "Explaining and Harnessing Adversarial Examples." [Reference Link](#)
- OWASP. "Fuzzing AI Systems: Techniques and Tools." [Reference Link](#)
- <https://pmc.ncbi.nlm.nih.gov/articles/PMC10535982/>

ACRONYMS

DAST: Dynamic Application Security Testing – A method of testing the security of an application by examining it in its running state for vulnerabilities.

RCE: Remote Code Execution – A security vulnerability that allows an attacker to run arbitrary code on a remote machine.

SAST: Static Application Security Testing – A method of analyzing source code or compiled applications for vulnerabilities without executing the code.

SDLC: Software Development Life Cycle – A process for planning, developing, testing, and deploying software systems, typically involving stages like requirements, design, coding, testing, and maintenance.

XSS: Cross-Site Scripting – A type of security vulnerability where an attacker injects malicious scripts into web pages viewed by others.